Q1:

Implement Classes Screen and Window_mgr in a single .cpp file following the specification given below.
Goal: Model a two-dimensional text screen using a one-dimensional string and manage multiple screens through a separate window manager class.

Part A: Screen Design
Represent the screen as a contiguous string named contents.
Each character in the string represents one cell on a rectangular screen.
Store height and width separately.
Never store a 2D array. The screen must be reconstructed logically from the dimensions and the string.
Maintain a cursor indicating the current position (or index) within the contents string.
Class invariant: contents.size() == height * width.
Using the above description, identify the class member attributes and define them.

Part B: Coordinate Mapping Logic
Convert a row and column into a string index using: (row * width) + column. Here, row and column start from 0.

All row/column operations must use this mapping.
Example: width=5, row=2, col=3 -> index=13.
Public getter and setter methods get(row,col) and set(row,col,ch) must use the same mapping formula.

Part C: Constructors
Provide a default constructor that creates an empty or default screen. The default height and width are 20 and 50 respectively. Cursor is at position 0 (0th row and 0th col) by default. Default fillCharacter is "whitespace", i.e. the contents should be 20*50 = 1000 character long string of whitespace.
Provide a constructor Screen(height,width,fillCharacter).
The constructor must allocate height*width characters and initialize every position with fillCharacter.
Create a copy constructor (to deep copy) as well for the Screen class, taking one Screen& object.

Part D: Member Function Specifications
get(): return the character currently under the cursor.
get(row,col): compute location and return that character.

move(row,col): compute location and update cursor. Return a reference to the current object.
set(ch): replace the character at the current cursor position.
set(row,col,ch): compute location and replace the appropriate character. If row and col are outside the screen width and height ranges, ignore the input, do nothing, silently return.
display(ostream& os): print contents of the screen using os <<.

Part E: Chaining Requirements
The following style must work: screen.move(4,0).set("#").display(cout) conceptually.
Return *this from mutating members (member methods which modify class attributes).
No temporary objects should be created merely to support chaining.

Part F: Window_mgr Design
Window*mgr owns multiple Screen objects inside an array (max 10 screens) in form of composition. That means, when Window*mngr object dies, all the screens that it owns, also die.
The array of screen is of objects (not pointers), so be sure to remove any copies, if they exist, to model accurate composition.
By default, Window_mgr has one default screen already stored.
addScreen(Screen): append a screen to the array and return its index. (if you are coping the object,make sure to remove the original copy)
addScreen(Screen*): overloaded add method which takes pointer, and performs the same operation of adding the screens.
Each screen is identified by its index in the array.

Part G: Friendship
Window_mgr must access the private contents member of Screen.
Grant friendship explicitly rather than exposing public setters for implementation details.
Do not make all members public.

Part H: clear() Logic
clear(index) retrieves the target screen.
Then reassign the contents string directly (without using getter/setter of the screen class) with the default string value (whitespaces repeated height*width number of times).
Dimensions and cursor remain valid after clearing.
Only the displayed characters change.

Required Driver Demonstration (main):
Create at least three different screens.
Display screens before modification.

Use move and set to edit contents.
Add screens to Window_mgr.
Clear one screen and show the effect.
Demonstrate const display and chained calls.

Q2: Representing Sales Data as a class (Submit a Sales*data.h and Sales*_data.cpp file. The header file is only for class declaration. The .cpp file should have class implementation as well as main. Do not submit a separate file for main.)

Design a business object that represents aggregate sales information for a book and supports construction, output, and transaction accumulation.

Part A: Class Sales_data
Store bookNo (unique identifier for a book), units_sold, and revenue as private data members.
Book No is stored as a string variable, which represents the real world ISBN -- International Standard Book Number; units_sold is unsigned int (because it cannot be negative), and revenue is a double variable.
Revenue represents total money earned by selling units*sold number of that particular bookNo. Therefore, they are specific to a particular Sales_data object.*
Revenue and units sold together are used to compute average selling price of that bookNo.

Have getter isbn() for getting the bookNo.

Part B: Business Rules
Transactions can be combined only when they refer to the same ISBN.
The class does not store every sale individually; it stores aggregate totals.
Revenue must always remain consistent with recorded sales.

Part C: Constructor Specifications
Default constructor initializes all members to an empty/zero state.
ISBN-only constructor initializes identifier and leaves totals at zero.
Three-parameter constructor takes ISBN, quantity, and price.
Revenue calculation logic: revenue = quantity * price.

Part D: combine() Logic

This is a method to combine two sales data.
Accept another Sales_data object.
Add units_sold from the argument to the current object.
Add revenue from the argument to the current object.
Return *this so additional operations may be chained.
Assume ISBN compatibility before combining, i.e. if combine() is called when the callee and caller ISBN numbers are different, no action should be taken and the method should simply return the caller object without modifying anything.

Part E: avg_price() Logic
This is a private method, acting as a helper.
Average price is revenue divided by units_sold.
When units_sold equals zero, return zero instead of dividing.
This helper should not be visible to client code.

Part F: Nonmember Functions (Global Friend)
Overloaded operator << (using friendship to make cout << work for this class, as discussed in class) to print ISBN, units sold, revenue, and average price using cout object.
Follow this exact specific format: <ISBN> <units*sold*> <*revenue*> <*avg*_price>\n.
add(lhs,rhs): copy lhs, invoke combine(rhs), return the combined object. It directly returns the object obtained from the call to combine.

Part G: Information Hiding
Users must never modify revenue directly.
All data members remain private.
External code interacts through constructors, combine(), isbn(), print using <<, and add().

Driver code for testing (main):
Construct objects using every constructor.
Read at least five records from input.
Combine compatible transactions.
Verify average price manually for at least one example.
Test zero-sales behavior.
Show that add() does not modify its arguments.